

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 3, Sesión 1 (Sesión 5)

Repaso y Contexto

Funciones en Python

Parámetros y Alcance

Módulos y Paquetes

Ejercicios Guiados

Soluciones de Referencia

Actividad Práctica

Conclusiones

Repaso y Contexto

- **Semana 2, Sesión 4 (Sesión 4)** se centró en:
 - Laboratorio práctico con estructuras de control (**if**, **while**).
 - Problemas tipo: “Adivina el número”, “Calculadora de Calificaciones”, “Movimiento Discreto”.
 - Manejo de validaciones, contadores y salidas controladas de bucles.

- **Comprender** la sintaxis y concepto de funciones en Python (**def**, parámetros, **return**).
- **Explorar** cómo organizar código en módulos y paquetes simples.
- **Aplicar** estos conceptos a pequeños proyectos para mejorar reutilización y claridad.
- **Conectar** funciones con los temas previos (estructuras de control, manejo de datos, etc.).

Funciones en Python

- **Reutilización de Código:** evitar escribir la misma lógica en múltiples lugares.
- **Organización:** encapsular tareas específicas en “cajas negras”.
- **Legibilidad:** el nombre de la función describe la operación que realiza.
- **Mantenibilidad:** cambiar la lógica en un solo lugar (la definición de la función).

```
1 def nombre_de_funcion(param1, param2, ...):  
2     """  
3     Documentación opcional (docstring).  
4     Explica qué hace la función, los parámetros y el retorno.  
5     """  
6     # bloque de código  
7     # opcionalmente retornar un valor  
8     return resultado
```

- **Parámetros:** valores de entrada que la función utiliza.
- **return:** finaliza la función y opcionalmente devuelve un valor.
- **Mejor práctica:** Usar nombres descriptivos para parámetros y funciones.

Funciones en Python

Ejemplo: Función para Calcular el Área de un Círculo

```
1 import math
2
3 def area_circulo(radio):
4     """
5     Retorna el área de un círculo de radio 'radio'.
6     Formula:  $\pi * r^2$ 
7     """
8     area = math.pi * (radio**2)
9     return area
10
11 # Uso de la función
12 r = 5
13 a = area_circulo(r)
14 print("El área del círculo es:", a)
```

- Encapsulamos la operación en `area_circulo`.
- Parámetro `radio`.
- Retorno con `return area`.

Parámetros y Alcance

```
1 def calcular_salario(base, horas_extra=0):
2     """Calcula salario sumando base + 2000 por hora extra"""
3     return base + 2000 * horas_extra
4
5 s1 = calcular_salario(50000)      # horas_extra por defecto: 0
6 s2 = calcular_salario(50000, 3)  # 3 horas extra
7 print(s1, s2)
```

- Parámetros con valor por defecto (ej. `horas_extra=0`).
- Posicionales: deben ir en orden; `calcular_salario(50000, 3)`.
- Keyword arguments: `calcular_salario(base=50000, horas_extra=3)`.

Parámetros y Alcance

Scope o Alcance de Variables

- **Local:** Variables definidas dentro de la función solo existen dentro de ella.
- **Global:** Variables definidas fuera de cualquier función son accesibles dentro, pero no se recomienda modificarlas sin razón.
- **Mejor práctica:** Mantener funciones “puras”, evitando depender de variables globales.

Ejemplo: `def f(): x=10` — la variable `x` vive solo en `f()`.

Parámetros y Alcance

Conflicto Global vs Local

```
1 x = 5 # global
2
3 def foo():
4     x = 10 # local
5     print("Dentro de foo, x =", x)
6
7 foo()
8 print("Fuera de foo, x =", x)
```

- Impresiones:
 - "Dentro de foo, x = 10"
 - "Fuera de foo, x = 5"
- **Evitar confusiones** usando nombres significativos y parámetros claros.

Módulos y Paquetes

- Un **módulo** es un archivo **.py** que contiene definiciones de funciones, variables y clases.
- **Ventaja:** podemos importar este archivo desde otros scripts y reutilizar el código.
- **Ejemplo:** Crear un archivo **mifunciones.py** con varias funciones y luego:

```
import mifunciones
```

Módulos y Paquetes

Ejemplo de Módulo: *mifunciones.py*

```
1 # mifunciones.py
2
3 def suma(a, b):
4     return a + b
5
6 def resta(a, b):
7     return a - b
```

Uso en otro archivo:

```
1 import mifunciones
2
3 print(mifunciones.suma(3, 4))
4 print(mifunciones.resta(10, 2))
```

Módulos y Paquetes

Importaciones Específicas

```
1 from mifunciones import suma
2
3 resultado = suma(5, 7)
4 print(resultado)
```

- Trae solo la función **suma** del módulo.
- **Cuidado:** peligro de colisiones de nombre (si hay otra **suma**).
- **from mifunciones import *** trae todo, también puede causar conflictos.

Módulos y Paquetes

Paquetes en Python

- Un **paquete** es una carpeta que contiene un archivo `__init__.py` y varios módulos `.py`.
- Estructura típica: `mypackage/modulo1.py`
`mypackage/modulo1.py`
- Para usarlo: `import mipaquete.modulo1`.

Uso: Organizar proyectos grandes en submódulos lógicamente separados.

Ejercicios Guiados



Enunciado

Crear una función que reciba dos números y realice lo siguiente:

- Si ambos números son positivos, retornar su suma.
- Si uno de los números es negativo, retornar su división.
- Si ambos números son negativos, retornar su producto.

Conceptos: Uso de estructuras condicionales (`if`, `elif`, `else`).



Enunciado

Crear una función que reciba una lista de números y clasifique cada número como:

- Positivo.
- Negativo.
- Cero.

La función debe imprimir el resultado para cada número y retornar la cantidad de números positivos, negativos y ceros.

Conceptos: Uso de ciclos `for` y estructuras condicionales.



Enunciado

Crear una función que reciba un número entero n y realice lo siguiente:

- Solicitar al usuario n números enteros.
- Validar que cada número ingresado sea mayor que cero. Si no lo es, volver a solicitar el número.
- Retornar la suma de los n números ingresados.

Conceptos: Uso de ciclos `while` y validaciones.



Enunciado

Crear una función que reciba un número entero n y retorne los primeros n términos de la serie de Fibonacci.

- La serie de Fibonacci comienza con 0 y 1.
- Cada término siguiente es la suma de los dos anteriores.

Conceptos: Uso de ciclos **for** y listas.



Enunciado

Crear una función que reciba un número entero y determine si es primo o no.

- Un número primo es divisible únicamente por 1 y por sí mismo.
- La función debe retornar **True** si el número es primo y **False** en caso contrario.

Conceptos: Uso de ciclos **for** y estructuras condicionales.

Soluciones de Referencia

Soluciones de Referencia

Solución 1 de Referencia:

Suma Condicional



```
1 def suma_condicional(a, b):
2     """
3     Realiza operaciones según los valores de a y b:
4     - Si ambos son positivos, retorna su suma.
5     - Si uno es negativo, retorna su división.
6     - Si ambos son negativos, retorna su producto.
7     """
8     if a > 0 and b > 0:
9         return a + b
10    elif a < 0 and b < 0:
11        return a * b
12    else:
13        return a / b
14
15 # Ejemplo de uso
16 resultado = suma_condicional(5, -3)
17 print("Resultado:", resultado)
```

Discusión: ¿Qué sucede si $b = 0$ en la división? ¿Cómo manejar este caso?

Soluciones de Referencia

Solución 2 de Referencia:

Clasificación de Números



```
1 def clasificar_numeros(lista):
2     """
3     Clasifica números en positivos, negativos y ceros.
4     Retorna la cantidad de cada tipo.
5     """
6     positivos = 0
7     negativos = 0
8     ceros = 0
9
10    for num in lista:
11        if num > 0:
12            print(f"{num} es positivo")
13            positivos += 1
14        elif num < 0:
15            print(f"{num} es negativo")
16            negativos += 1
17        else:
18            print(f"{num} es cero")
19            ceros += 1
20
21    return positivos, negativos, ceros
22
23 # Ejemplo de uso
24 resultado = clasificar_numeros([3, -1, 0, 7, -5])
25 print("Positivos, Negativos, Ceros:", resultado)
```

Discusión: ¿Cómo manejar listas vacías o valores no numéricos?

Soluciones de Referencia

Solución 3 de Referencia: Suma de Números con Validación



```
1 def suma_con_validacion(n):
2     """
3     Solicita n números positivos al usuario y retorna su suma.
4     Valida que cada número ingresado sea mayor que cero.
5     """
6     suma = 0
7     contador = 0
8
9     while contador < n:
10         num = float(input(f"Ingrese el número {contador + 1}: "))
11         if num > 0:
12             suma += num
13             contador += 1
14         else:
15             print("El número debe ser mayor que cero. Intente de nuevo.")
16
17     return suma
18
19 # Ejemplo de uso
20 resultado = suma_con_validacion(3)
21 print("Suma total:", resultado)
```

Discusión: ¿Cómo manejar entradas no numéricas o interrupciones del usuario?

Soluciones de Referencia

Solución 4 de Referencia:

Serie de Fibonacci



```
1 def fibonacci(n):
2     """
3     Retorna los primeros n términos de la serie de Fibonacci.
4     """
5     if n <= 0:
6         return []
7     elif n == 1:
8         return [0]
9     elif n == 2:
10        return [0, 1]
11
12    serie = [0, 1]
13    for i in range(2, n):
14        siguiente = serie[-1] + serie[-2]
15        serie.append(siguiente)
16
17    return serie
18
19 # Ejemplo de uso
20 resultado = fibonacci(10)
21 print("Serie de Fibonacci:", resultado)
```

Discusión: ¿Cómo optimizar para valores grandes de n ?

Soluciones de Referencia

Solución 5 de Referencia:

Verificación de Número Primo



```
1 def es_primo(num):
2     """
3     Determina si un número es primo.
4     Retorna True si es primo, False si no.
5     """
6     if num < 2:
7         return False
8     for i in range(2, num):
9         if num % i == 0:
10            return False
11    return True
12
13 # Ejemplo de uso
14 resultado = es_primo(13)
15 print("¿Es primo?", resultado)
```

Discusión: ¿Cómo manejar números negativos o muy grandes?

Actividad Práctica

Actividad Práctica

Actividad Extra 1:

Función Avanzada



Enunciado

Crear una función que calcule el factorial de un número de forma recursiva (que la función se llame a sí misma).

Objetivo: Aplicar recursión.

Actividad Práctica

Actividad Extra 2:

Módulo Personalizado



Enunciado

Crear un módulo con funciones para calcular el área y perímetro de figuras geométricas básicas (círculo, cuadrado, triángulo).

Objetivo: Modularizar el código y practicar importaciones.

Actividad Práctica

Actividad Extra 3:

Proyecto Integrador



Enunciado

Diseñar un programa que utilice funciones y módulos para resolver un problema práctico, como una calculadora científica básica.

Objetivo: Integrar conceptos de funciones, módulos y estructuras de control.

- **Objetivo:** Organizar funciones útiles en un módulo y probar su importación.
- **Instrucciones:**
 1. Crea un archivo `utilidades.py` con al menos 3 funciones (ej. `factorial(n)`, `es_primo(n)`, etc.).
 2. En un **notebook de Colab** o un script `main.py`, importa `utilidades` y prueba dichas funciones.
 3. (Opcional) Añade una cuarta función con un **parámetro por defecto** (`def hola_mundo(nombre="Mundo")`), etc.).

Sugerencia: Comenta tu código y explica la lógica de cada función.

Reflexión: ¿Cómo ayuda la modularización a proyectos más grandes?

Actividad Práctica

Trabajo Colaborativo

- Formar **parejas o tríos**.
- Diseñar **un pequeño módulo** de funciones:
 - Matemáticas, Estadística básica, Conversión de unidades, etc.
- Utilizar dichas funciones en **otro script** o **notebook**.
- **Opcional:** Explorar la creación de una carpeta con `__init__.py` para armar un paquete simple.

Actividad Práctica

Espacio para Dudas y Apoyo

- ¿Problemas al importar el módulo en Colab?
- ¿Cómo organizar los archivos en Google Drive?
- ¿Errores de `ModuleNotFoundError`?

Consulta en voz alta o pide ayuda a tus compañeros.

- ¿Se entendió la **separación** entre la lógica (en un módulo .py) y el código principal?
- ¿Ventajas de tener todo en un solo archivo vs. múltiples módulos?
- ¿Dudas sobre parámetros y valores por defecto?

Conclusiones

- Apreciamos la **modularización** del código para mejor legibilidad y mantenimiento.
- Vimos **funciones**: sintaxis **def**, parámetros, **return**, docstring.
- Exploramos la creación de **módulos** y su importación en otros scripts o notebooks.
- Ahora estamos preparados para proyectos más grandes y ordenados.

- **Sesion 8 (Semana 4):** Continuaremos con el uso de módulos y paquetes, y profundizaremos en el uso de **pip** y librerías externas.
- **Tarea sugerida:**
 - Crear un pequeño proyecto con un paquete **mipaquete/** y varios módulos (ej. **mipaquete/calculos.py**, **mipaquete/utiles.py**, etc.).
 - Probar importarlos y usarlos en un script principal.

¡Gracias y hasta la próxima sesión!

- Guarda tus **notebooks** y archivos **.py** en Google Drive.
- Explora la documentación oficial de Python (sobre **funciones** y **módulos**).
- ¡Sigue practicando!